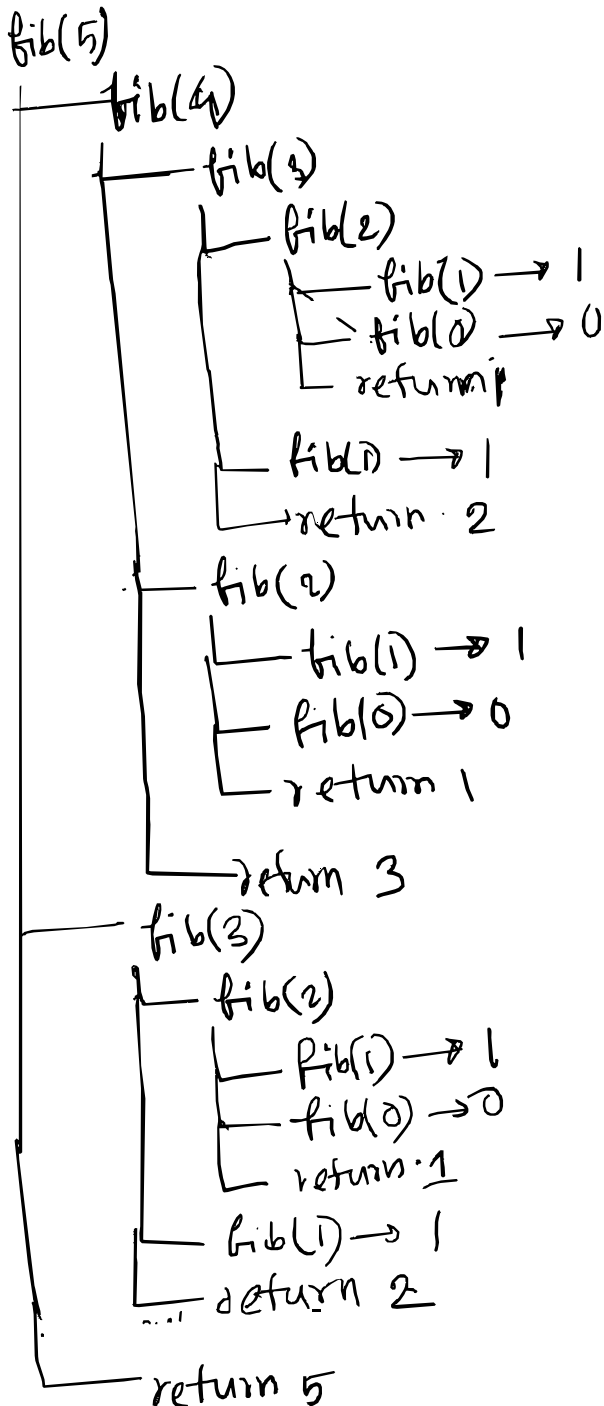


Handson #.04

Problem 0

The recursion unfolding is shown below:



Problem 1

① Code uploaded to github.

②

- We insert the 1st element of each of K arrays into the min-heap
- Each insertion into the heap takes $O(\log K)$ time.
- We insert K elements, hence, $O(K \log K)$
- There are total $N \times K$ elements across all arrays.
- Each extraction takes $O(\log K)$, and after each extraction, we insert the next element from the same array into the heap.
- ~~hence~~ the number of heap operation is $O(N \times K \log K)$

Hence, the total time complexity

$$= O(k \log k) + O(N \times k \log k)$$

$$= O(N \times k \log k)$$

$$\therefore T(n) = O(Nk \log k)$$

③ We can improve it by merging the two arrays at a time in a divide and conquer system.

This would also achieve a time complexity of $O(Nk \log k)$, but will have a lower constant factor.

Problem 2

① Code uploaded to github Repo.

② Here, the function removes duplicates using two pointer approach.

- The loop iterates once over the array of size N , making the number of operations proportional to N .
- The total number of operations is $O(N)$ each iteration taking $O(1)$ time.

hence, the overall time complexity,

$$T(n) = O(N)$$

③ we could use parallel processing.

If the array is very large, we can divide it into segments and process them in parallel. A multi-threaded approach could improve performance. It will ensure faster executions on multi-core CPUs.